LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamualaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga Laporan Akhir Praktikum Pemrograman Berorientasi Objek ini dapat diselesaikan dengan baik. Shalawat serta salam semoga senantiasa tercurahkan kepada Nabi Muhammad SAW.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum Pemrograman Berorientasi Objek yang membahas konsep polimorfisme, penimpaan method (method overriding), pengikatan dinamis (dynamic method dispatch), serta penggunaan parameter polimorfik pada fitur transfer antar rekening menggunakan bahasa pemrograman Java.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Pemrograman Berorientasi Objek yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 24 September 2026

Sofian Arba’i

BAB I
Pekan 1 (Class, Object, Constructor dan Method)

1.1 Pemahaman Teori

Class sebagai Blueprint, Menurut Deitel & Deitel (2017) dalam bukunya Java: How to Program, sebuah class adalah cetak biru (blueprint) dari sebuah objek. Sierra & Bates (2005) dalam Head First Java mengilustrasikan bahwa dua objek dari class yang sama misalnya Rekening dapat memiliki nilai state seperti saldo yang sama sekali berbeda, dan perubahan pada satu objek tidak akan memengaruhi objek lainnya. Constructor adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya adalah untuk menginisialisasi state awal dari objek tersebut (Oracle, 2023).

1.2 Langkah-Langkah dan Tugas

Berikut langkah-langkah pada praktikum pekan 1:

1. Buat class Rekening
2. Tambahkan State/Atribut
3. Buat Constructor
4. Buat Method/Behavior

Class Rekening memiliki tiga atribut, yaitu nomorRekening dan namaPemilik yang bertipe String, serta saldo yang bertipe double. Memiliki konstruktor Rekening (String nomor, String nama, double saldoAwal). Selanjutnya, class ini memiliki tiga buah method. Method setorTunai (double nominal), Method tarikTunai (double nominal), Terakhir, method cekInformasi (). Kode program lengkap class Rekening sebagai berikut.

5. Membuat Engine CLI pada Main.java

Pada awal program dibuat objek Scanner untuk membaca input, variabel akunAktif bertipe Rekening bernilai null (belum ada rekening), dan variabel

boolean isRunning sebagai kondisi perulangan while yang menampilkan menu utama selama bernilai true. Pilihan menu divalidasi dengan input.hasNextInt() agar berupa angka, lalu switch-case mengarahkan alur sesuai pilihan. Pada case 1, program meminta nomor rekening, nama pemilik, dan saldo awal, lalu membuat objek Rekening dengan new yang otomatis menjalankan konstruktor dan menyimpannya ke akunAktif. Pada case 2, 3, dan 4, program memeriksa apakah akunAktif null: jika ya, tampil pesan error, jika tidak, dipanggil setorTunai(), tarikTunai(), atau cekInformasi(). Case 0 mengubah isRunning menjadi false sehingga perulangan berhenti. Secara keseluruhan, Main menunjukkan cara objek Rekening dibuat dan digunakan lewat pemanggilan method.

Tugas pada praktikum pekan 1

1. Implementasi Fitur Tarik Tunai

Pada tugas 1, diminta menambahkan method tarikTunai(double nominal) ke dalam class Rekening dengan aturan saldo tidak boleh menjadi minus dan nominal penarikan minimal 10.000. Jika nominal kurang dari 10.000 atau melebihi saldo, program tidak boleh crash, melainkan menampilkan pesan "Transaksi Gagal: Minimal nominal penarikan 10.000" atau "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]". Method ini kemudian diterapkan pada menu 3 di class Main

2. Tantangan Multi-Akun

Pada tugas 2 (tantangan multi-akun), penyimpanan akun di Main.java yang semula hanya satu variabel akunAktif diubah menggunakan ArrayList, sehingga setiap rekening baru yang dibuat melalui menu "Buka Rekening Baru" ditambahkan ke dalam daftar dan akun lama tidak tertimpa. Sebagai bonus, ditambahkan menu "Ganti Akun" yang meminta nomor rekening, mencarinya di dalam ArrayList, lalu menjadikan objek yang cocok sebagai akunAktif.

1.3 Link Repository

Berikut merupakan link repository github untuk praktikum PBO pekan 1.
sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-1

BAB II
Relasi Antar Objek (Class Diagram & Asosiasi)

1. Pemahaman Teori

Diagram kelas atau class diagram adalah jenis diagram statis yang menunjukkan banyak komponen sistem secara grafis. Umumnya para pengembang menggunakannya untuk menguraikan bagaimana sebuah sistem dapat terlihat dalam bentuk statis. Biasanya, para pengembang atau developer menggunakan kelas diagram dalam menggambarkan bagaimana bentuk statis sebuah sistem ketika menciptakan software.

2. Langkah-Langkah dan Tugas

Berikut langkah-langkah pada praktikum pekan 2:

1. Membuat Class Diagram Transaksi

   Class Transaksi memiliki 3 buat atribute yaitu idTransaksi, jenis dan nominal. Jenis transaksi ada 2 buah yaitu Debit digunakan untuk penarikan dan Kredit digunakan untuk penyetoran. Membuat method cetakDetail() yang nantinya akan digunakan untuk menampilkan 1 baris detail transaksi.

2. Membuat Relasi pada Rekening.java

   Menambahkan atribut baru yang merepresentasikan relasi has-a yaitu satu rekening memiliki banyak transaksi.

3. Mencatat Riwayat Transaksi

Tugas pada praktikum pekan 2 yaitu, Tugas 1 : Integrasi Rekam Jejak Penarikan dan Tugas 2: Fitur Cetak Mutasi Rekening.

3. Link Repository

Berikut merupakan link repository github untuk praktikum PBO pekan 2. sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-2

BAB III
Keamanan & Integritas Data (Enkapsulasi)

3.1 Pemahaman Teori

Pada modul sebelumnya, semua atribut class tidak memiliki modifier, sehingga menggunakan aturan default yaitu dapat diakses langsung dari luar class. Hal ini sangat berbahaya. Jika tidak dikunci, sistem tidak akan bisa mencegah manipulasi langsung seperti akunAktif.saldo = 9999999.

1. Information Hiding: Robert C. Martin (2008) dalam bukunya Clean Code menekankan behavior (method), bukan mengekspos data (variabel) secara langsung. akses data, antara lain: bahwa kita harus menyembunyikan detail implementasi internal. Objek harus mengekspos
2. Access Modifiers di Java: Java menyediakan kata kunci untuk mengatur tingkat visibilitas akses data, antara lain:
   - private: Akses paling ketat. Data atau method hanya bisa diakses oleh class itu sendiri. Ini adalah aturan emas untuk melindungi state atau atribut kelas.
   - protected: Akses menengah. Data atau method dapat diakses oleh class dalam package yang sama, serta oleh subclass meskipun berada di package yang berbeda. Ini sangat berguna ketika kita menerapkan hierarki Inheritance (Pewarisan).

- public: Akses paling longgar. Bisa diakses dari mana saja tanpa batasan. Umumnya digunakan untuk method layanan yang memang perlu dipanggil dari luar.

3. Getter dan Setter: Oracle (2023) menyebutkan bahwa akses ke variabel private harus difasilitasi melalui method public khusus. Ini memungkinkan kita menyisipkan logika validasi seperti cek panjang PIN sebelum variabel diproses.

3.2 Langkah-Langkah dan Tugas

Berikut langkah-langkah pada praktikum pekan 3:

1. Mengunci Entitas Transaksi: Buka file Transaksi.java. Kita akan mengubah variabelnya menjadi private dan memberikan Getter hanya untuk data yang perlu dibaca. Objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, sehingga kita tidak membuat Setter.
2. Mengunci Entitas Rekening dan Menambahkan PIN: mengamankan saldo dan menambahkan atribut PIN.

Tugas pada pekan 3 yaitu, Tugas: Mengamankan CLI dengan Otentikasi, program Main.java diminta diperbarui agar mematuhi aturan keamanan baru berbasis otentikasi PIN. Pertama, pada menu 1 (Buka Rekening), pengguna harus memasukkan PIN berupa String angka 6 digit sebelum objek rekening baru dibuat. Kedua, transaksi pada menu 3 (Tarik Tunai) dan menu 6 (Cetak Mutasi) harus diamankan: sebelum kedua method tersebut dijalankan, sistem meminta pengguna memasukkan PIN, lalu memanggil akunAktif.otentikasi(pinYangDiinput).

3.3 Link Repository

Berikut merupakan link repository github untuk praktikum PBO pekan 3. sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-3

BAB IV
Ekspansi Produk Keuangan (Inheritance)

4.1 Pemahaman Teori

Seiring berkembangnya sistem perbankan, bank tidak hanya memiliki satu jenis rekening. Ada "Rekening Tabungan" yang mana akan mendapat bunga bulanan dan "Rekening Giro" yang memiliki fasilitas pinjaman atau overdraft. Jika kita membuat class baru dari nol untuk masing masing entitas tersebut, kita akan melanggar prinsip DRY (Don't Repeat Yourself) karena menduplikasi banyak kode logika PIN dan mutasi.

- Konsep Pewarisan (Inheritance): Menurut Deitel dan Deitel (2017), pewarisan adalah bentuk software reuse di mana class baru diciptakan dengan menggunakan atribut dan behavior dari class yang sudah ada, lalu menambahkan kapabilitas baru. Hubungan ini dikenal sebagai relasi "Is-A" contohnya Tabungan is-a Rekening.
- Penggunaan Keyword extends dan super: Di Java, subclass mewarisi kelas induk menggunakan kata kunci extends. Sementara itu, kata kunci super berfungsi sebagai jembatan langsung ke kelas induk.
  - super() digunakan di dalam konstruktor kelas anak untuk mengeksekusi inisialisasi konstruktor induk. Pemanggilan ini wajib berada di baris paling pertama di dalam konstruktor anak (Oracle, 2023).
  - super.namaMethod() digunakan jika kelas anak perlu memanggil metode asli milik induk yang mungkin telah dimodifikasi (di-override).
- Pentingnya protected: Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.
- Upcasting (Implisit): Menyimpan objek kelas anak ke dalam variabel referensi kelas induk contohnya Rekening akun = new RekeningTabungan(...). Proses ini otomatis dan aman. Fungsinya sangat krusial agar kita bisa menyimpan berbagai jenis produk rekening ke dalam satu koleksi yang seragam seperti ArrayList tipe Rekening.

- Downcasting (Eksplisit): Memaksa referensi kelas induk kembali menjadi kelas anak spesifik contoh RekeningTabungan tab = (RekeningTabungan) akun. Proses ini rawan memicu error ClassCastException saat program berjalan jika objek aslinya ternyata bukan Tabungan. Oleh karena itu, downcasting wajib didahului oleh pengecekan tipe menggunakan operator instanceof.

4.2 Langkah-Langkah dan Tugas

Berikut langkah-langkah pada praktikum pekan 4:

1. Memastikan Akses Modifier di Rekening.java: Buka kembali file Rekening.java. Pastikan atribut saldo dan riwayatTransaksi menggunakan hak akses protected, sementara pin dan nomorRekening tetap private.
2. Membuat Subclass RekeningTabungan: Buat file baru bernama RekeningTabungan.java. Produk tabungan memiliki fitur tambahan yaitu Suku Bunga Bulanan.
3. Membuat Subclass RekeningGiro: Buat file baru bernama RekeningGiro.java. Produk Giro digunakan pebisnis dan memiliki fitur tambahan yaitu Batas Overdraft (Limit Pinjaman).

Tugas pada pekan 4 yaitu, memperbarui antarmuka CLI di Main.java agar mendukung produk-produk baru ini secara interaktif. Tugas 1: Pembuatan Multi-Produk, setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis". Tugas 2: Simulasi Waktu Akhir Bulan, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

4.3 Link Repository

Berikut merupakan link repository github untuk praktikum PBO pekan 4. sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-4

BAB V
Transaksi Dinamis (Polymorphism)

5.1 Pemahaman Teori

Kata Polymorphism berasal dari bahasa Yunani yang berarti "memiliki banyak bentuk". Dalam PBO, polimorfisme memungkinkan satu antarmuka menggunakan nama method yang sama untuk dieksekusi dengan cara yang berbeda-beda tergantung pada tipe objek aslinya pada saat runtime / waktu eksekusi.

1. Method Overriding (Runtime Polymorphism): Eckel (2006) dalam *Thinking in Java* menjelaskan bahwa overriding terjadi ketika subclass mendefinisikan ulang method milik kelas induknya. Method tersebut harus memiliki nama dan parameter yang persis sama dengan method di kelas induk. Jika objek diinstansiasi dari kelas anak, Java akan selalu mengeksekusi versi method milik kelas anak.
2. Dynamic Method Dispatch (Late Binding): Menurut Oracle (2023), JVM menentukan method mana yang akan dipanggil pada saat program berjalan, bukan pada saat kompilasi. Jika kita memanggil akunAktif.tarikTunai(), JVM akan melihat apakah akunAktif itu berisi Tabungan biasa atau Giro, lalu memanggil logika yang tepat tanpa kita harus menuliskan struktur if-else atau switch-case.
3. Anotasi @Override: Digunakan di Java sebagai praktik terbaik (best practice) untuk memberi tahu compiler bahwa kita berniat menimpa method induk. Jika terjadi kesalahan ketik pada nama method, compiler akan memberikan pesan eror (Sierra dan Bates, 2005).

5.2 Langkah-Langkah dan Tugas

Berikut langkah-langkah pada praktikum pekan 5:

1. Refaktor Method tarikTunai pada Kelas Induk (Rekening.java): Agar fungsi penarikan dapat digunakan sebagai bagian dari fitur transfer nanti, kita perlu mengubah tipe kembalian (return type) tarikTunai dari void menjadi boolean yaitu mengembalikan true jika sukses, false jika gagal.
2. Melakukan Overriding pada RekeningGiro.java: Membuka file RekeningGiro.java dan menambahkan method tarikTunai yang menimpa (override) method milik kelas induk Rekening, dengan menggunakan anotasi @Override. Method hasil overriding ini memakai aturan batasOverdraft.
3. Uji Coba Pengikatan Dinamis Jalankan Main.java: Buat akun baru dan Pilih Produk: 2. Giro. Beri saldo 100.000 dan batas overdraft 500.000. Coba tarik tunai 400.000. saldo akan menjadi minus 300.000. inilah yang disebut Dynamic Dispatch.

Tugas pekan 5 yaitu, Tugas 1: Buat Method Transfer: Di dalam class Rekening.java, tambahkan satu method baru dengan nama: public void transfer (Rekening tujuan, double nominal, String pinInput). Tugas 2: Integrasi ke CLI Utama, Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.

5.3 Link Repository

Berikut merupakan link repository github untuk praktikum PBO pekan 5. sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-5

BAB VI
PENUTUP

6.1 Kesimpulan

Berdasarkan praktikum Pemrograman Berorientasi Objek selama lima pekan yang diterapkan pada sistem perbankan mini berbasis Java, dapat disimpulkan sebagai berikut.

1. Class, Object, Constructor, dan Method (Pekan 1). Class Rekening berfungsi sebagai blueprint yang menyimpan atribut dan method, sedangkan objek dibuat dengan kata kunci new yang otomatis menjalankan constructor. Penggunaan ArrayList memungkinkan program menyimpan banyak rekening tanpa saling menimpa.
2. Relasi Antar Objek (Pekan 2). Relasi asosiasi one-to-many diterapkan antara Rekening dan Transaksi, sehingga satu rekening dapat mencatat banyak transaksi pada ArrayList dan menampilkannya sebagai mutasi.
3. Enkapsulasi (Pekan 3). Atribut dikunci dengan private dan diakses melalui getter, sedangkan PIN diverifikasi lewat method otentikasi(). Dengan begitu data sensitif seperti saldo dan PIN tidak dapat diubah langsung dari luar class.
4. Inheritance (Pekan 4). RekeningTabungan dan RekeningGiro mewarisi Rekening dengan extends dan super(), sehingga logika PIN dan mutasi tidak ditulis ulang. Upcasting memungkinkan berbagai produk disimpan dalam satu ArrayList<Rekening>, dan downcasting yang didahului instanceof mengakses method khusus subclass dengan aman.
5. Polimorfisme (Pekan 5). Method tarikTunai di-override pada RekeningGiro, dan JVM memilih versi yang sesuai dengan tipe objek asli saat program berjalan (dynamic method dispatch). Hal ini terbukti dari penarikan Rp400.000 pada Giro bersaldo Rp100.000 yang menghasilkan saldo Rp-300.000 tanpa mengubah kode menu 3. Method transfer dengan parameter polimorfik Rekening juga dapat memproses transfer antar jenis rekening apa pun.

DAFTAR PUSTAKA

[1] Dani, D. (n.d.). *Pahami Konsep Association, Aggregation, Composition pada Java*. Medium. [Daring]. Tersedia pada: Class Diagram: Pengertian, Simbol, Manfaat beserta Contohnya. Diakses 18 September 2026.

[2] Deitel, P., & Deitel, H. (2017). *Java: How to Program, Early Objects* (11th ed.). Pearson Education.

[3] Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.

[4] ITBOX. (n.d.). *Class Diagram: Pengertian, Simbol, Manfaat beserta Contohnya*. [Daring]. Tersedia pada: Pahami Konsep Association, Aggregation, Composition pada Java | by Dimas Dani | Medium Diakses 18 September 2026.

[5] Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

[6] Oracle. (2023). *The Java™ Tutorials*. [Daring]. Tersedia pada: https://docs.oracle.com/javase/tutorial/